

The Harness Era Survey

A Critical Survey of LLM-Driven Vulnerability Research



A consolidated review of October 2024 through May 2026: nineteen months in which agentic vulnerability discovery moved from laboratory demonstration to production deployment, framed against the offensive capability shock of Anthropic's Mythos Preview and the defensive engineering response of Microsoft's MDASH.



Compiled by: Douglas Mun with AI Assistance

Classification: TLP:CLEAR - Public, Unrestricted Distribution

May 2026

Table of Contents

Abstract

List of Figures and Tables

1. Methodology

- 1.1 Scope and time window
- 1.2 Inclusion criteria for the landmark set
- 1.3 Inclusion criteria for the broader survey
- 1.4 Sources
- 1.5 Limitations of this survey

2. The Nineteen-Month Trajectory

- 2.1 The framing events
- 2.2 The convergent recipe

3. The Coverage Matrix: Demonstrating the Convergence

4. The Six Landmark Systems

- 4.1 Big Sleep — The Proof of Concept
- 4.2 AgentFlow — Harness Synthesis
- 4.3 VulnSage — Constraint Extraction and Reflection
- 4.4 The AlxCC Corpus — Competition-Scale Cyber Reasoning
- 4.5 A1 — Economic Actuality
- 4.6 MDASH — Production-Scale Defense

5. The Broader Research Landscape

- 5.1 Multi-agent discovery beyond the landmarks
- 5.2 Specialized harnesses that shaped the methodology
- 5.3 The defensive pipeline
- 5.4 Commercial products: the deployment layer
- 5.5 The agentic-fuzzing paradigm
- 5.6 Capability benchmarks (treated separately in Section 6)

6. Critical Assessment

- 6.1 Measurement validity: what the benchmark numbers don't tell you
- 6.2 What current LLMs actually struggle with
- 6.3 The Mythos disclosure: examining the strongest claims
- 6.4 Where the convergent recipe doesn't work

7. The Threat Model

- 7.1 Nation-state actors
- 7.2 Cybercriminal organizations
- 7.3 Independent actors and the "script kiddie" question
- 7.4 The defender side, similarly disaggregated

8. The Economics of AI Vulnerability Research

- 8.1 Per-finding cost across the surveyed systems
- 8.2 The bug-bounty market
- 8.3 Defensive deployment cost

9. Attacks on the Harnesses Themselves

- 9.1 Direct attacks on agent systems
- 9.2 Specific risks for vulnerability-discovery agents
- 9.3 Implication: defensive harnesses need defensive harnesses

10. The Mythos–MDASH Pairing, Reconsidered

- 10.1 What the pairing demonstrates
- 10.2 What the pairing does not yet demonstrate
- 10.3 The 6–12 and 12–18 month windows

11. Open Problems

12. Conclusion

References

Abstract

Between October 2024 and May 2026, LLM-driven vulnerability research underwent a methodological consolidation that this survey terms **the harness era**: the rapid convergence of independent research groups on a shared architectural pattern — specialized agents organized into a harness, fed by structured static-analysis input and runtime feedback channels, refined through reflection loops, and validated by external ground truth. We survey 28 systems across academia, industry, competition teams, and commercial products; profile six landmarks (Google's Big Sleep, AgentFlow, VulnSage, the AlxCC corpus, A1, and Microsoft's MDASH); and quantify the convergence via a coverage matrix showing that 86% of the surveyed systems implement all four recipe elements substantively. We then turn a critical lens on the field's reported capabilities, examining measurement validity, the gap between hand-curated success stories and average benchmark performance, the marketing-versus-capability question raised by Anthropic's Mythos disclosure, and the technical limits that remain unresolved — particularly LLMs' weaknesses in logic-bug detection and interprocedural reasoning. We argue that the binding constraint of the next phase is no longer model capability but deployment economics: whether MDASH-class defensive engineering becomes broadly available before Mythos-class offensive capability proliferates through open-weight diffusion. The paper closes with a detailed threat model that distinguishes nation-state, criminal, and individual actors, and an economic analysis that places per-finding compute cost in context with bug-bounty market dynamics.

List of Figures and Tables

Figures

- Figure 1: The nineteen-month trajectory (Section 2)
- Figure 2: The convergent recipe architecture (Section 2.2)
- Figure 3: Coverage heatmap across 28 systems (Section 3)
- Figure 4: Compute cost per finding (Section 8.1)

Tables

- Table 1: Coverage matrix of surveyed systems against the four recipe elements (Section 3)
- Table 2: Side-by-side comparison of the six landmark systems (Section 4)

1. Methodology

1.1 Scope and time window

This survey covers LLM-driven vulnerability discovery, exploit generation, and patch generation systems published or announced between **October 2024 and May 2026** — the nineteen months from Big Sleep's first publicly-confirmed AI-discovered zero-day to Microsoft's MDASH announcement. We include earlier work that anchors this period as background context, in particular Google Project Zero's Napttime publication of June 2024, which we identify as the design-principles document that the subsequent operational systems build on.

1.2 Inclusion criteria for the landmark set

We profile six systems as landmarks. Selection criteria, applied in order:

1. **Externally-validated real-world output:** assigned CVEs, vendor-confirmed disclosures, accepted patches, or bug-bounty acceptance. We exclude systems whose results are only benchmark scores.
2. **Methodological novelty:** each landmark must contribute a primitive that the subsequent literature has adopted — variant analysis (Big Sleep), harness synthesis (AgentFlow), structured constraint extraction (VulnSage), competition-scale CRSs (AlxCC), end-to-end smart-contract exploitation (A1), production-scale defensive deployment (MDASH).
3. **Coverage across the discovery-to-patch lifecycle:** the landmarks span discovery (Big Sleep), exploit generation (AgentFlow, VulnSage, A1), patch generation (AlxCC corpus), and production defense (MDASH).
4. **Transparency:** published technical details sufficient for independent assessment. Systems that report only headline numbers are profiled in the broader survey (Section 5) rather than as landmarks.

1.3 Inclusion criteria for the broader survey

The broader survey (Section 5) covers systems that contribute methodologically but do not satisfy the landmark criteria — typically because results are benchmark-only, vendor-confirmed but not yet disclosed, or because the system is one of several in a sub-genre (e.g., the proliferation of multi-agent pentest frameworks). We exclude pure capability evaluations and benchmarks; these are treated separately in Section 6.

1.4 Sources

We draw on arXiv preprints, peer-reviewed venues (USENIX Security, NDSS, IEEE S&P, ICSE, ICPC, ICLR), industry research blogs (Project Zero, Anthropic Red Team, Microsoft Security, Google DeepMind), competition disclosures (AlxCC archive, DEF CON 33), commercial vendor announcements (XBOW, Anthropic Claude Code Security), and independent third-party evaluations (UK AI Security Institute, CETaS analysis). We flag where claims rest on vendor self-reporting versus independent corroboration.

1.5 Limitations of this survey

Three limitations should be noted upfront. **First**, many of the most consequential results — particularly the Anthropic Mythos and Microsoft MDASH disclosures — are vendor-announced rather than peer-reviewed; we treat them as data points to be examined critically, not as established findings. **Second**, the field is moving faster than peer review; several systems we cite from arXiv may not survive replication. **Third**, the absence of negative results in the published literature creates a publication bias that this survey can document but not fully correct (Section 6.3).

2. The Nineteen-Month Trajectory

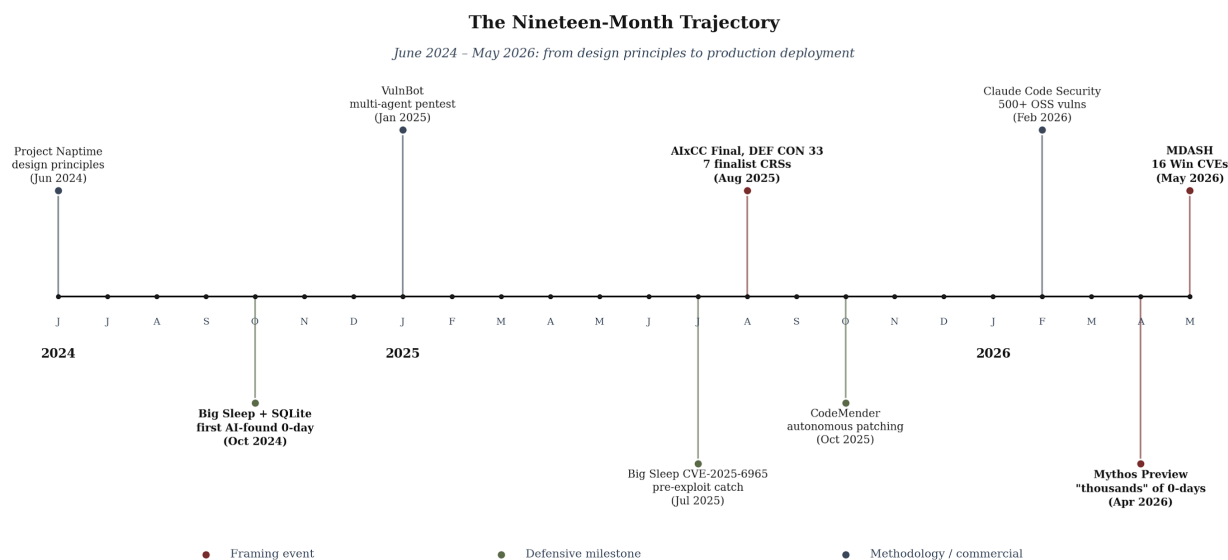


Figure 1. Timeline of landmark events, June 2024 – May 2026. Red markers indicate framing events (those that anchor the field's self-understanding); olive markers indicate defensive milestones; slate markers indicate methodology papers and commercial deployments. The nineteen-month window is bracketed by three publicly disclosed events — Project Naptime's publication of design principles, Big Sleep's first AI-discovered zero-day, and the Mythos–MDASH pairing of April–May 2026 — that anchor the field's self-understanding.

2.1 The framing events

The nineteen-month window covered by this survey is bracketed by three publicly disclosed events that anchor the field's self-understanding. We include a prologue event (Naptime, June 2024) as preceding context because the entire literature builds on it.

June 2024: Naptime (prologue). Project Zero's Sergei Glazunov and Mark Brand publish design principles for LLM-assisted vulnerability research, demonstrating up to 20× improvement on Meta's CyberSecEval 2 benchmark. The Naptime paper articulates the core insight that subsequent work converges on: LLMs need tool-grounded interaction with a target codebase rather than raw code-in-prompt analysis, and verification of agent output must be automatic rather than self-reported.

October 2024: Big Sleep finds the first AI-discovered zero-day. The Naptime framework, evolved into Big Sleep through collaboration between Project Zero and DeepMind, discovers an exploitable stack buffer underflow in SQLite via variant analysis. The bug is reported and patched the same day, before reaching a release. This is the proof of concept the field rallies around — and the formal start of the survey window.

April–May 2026: Mythos and MDASH. Anthropic disclosed on April 7, 2026, that Claude Mythos Preview had identified thousands of zero-day vulnerabilities across every major operating system and web browser, restricting access through Project Glasswing. Five weeks later, on May 12, 2026, Microsoft announces MDASH and ships 16 Windows CVEs in a single Patch Tuesday using a multi-model agentic scanning harness with more than 100 specialized agents. The pairing — capability shock and engineering response within thirty-five days — frames the central question of this paper.

2.2 The convergent recipe

Across the 28 systems surveyed, four architectural elements recur with striking consistency:

Element	Function	Failure mode is addressed
Specialized agents	Decompose vulnerability research into roles (analyst, generator, validator, debater, triager)	Context exhaustion, lost-in-the-middle, single-trace commitment
Structured static-analysis input	Pre-digest target into taint flows, call graphs, and exploit templates	Codebase-scale context overflow
Runtime feedback channels	Stream sanitizer reports, coverage maps, stderr, and execution traces back to agents	Pass/fail verdicts are an insufficient diagnostic signal
Reflection / retry loops	Iterate over execution feedback to refine agent outputs	LLM's inability to self-verify; sparse-reward optimization

We argue this recipe is not a stylistic preference but the unique structural fit for the problem. Section 3 develops this argument through coverage analysis of the surveyed systems; Section 4 profiles the systems that established each component; Section 6 examines where the recipe breaks down.

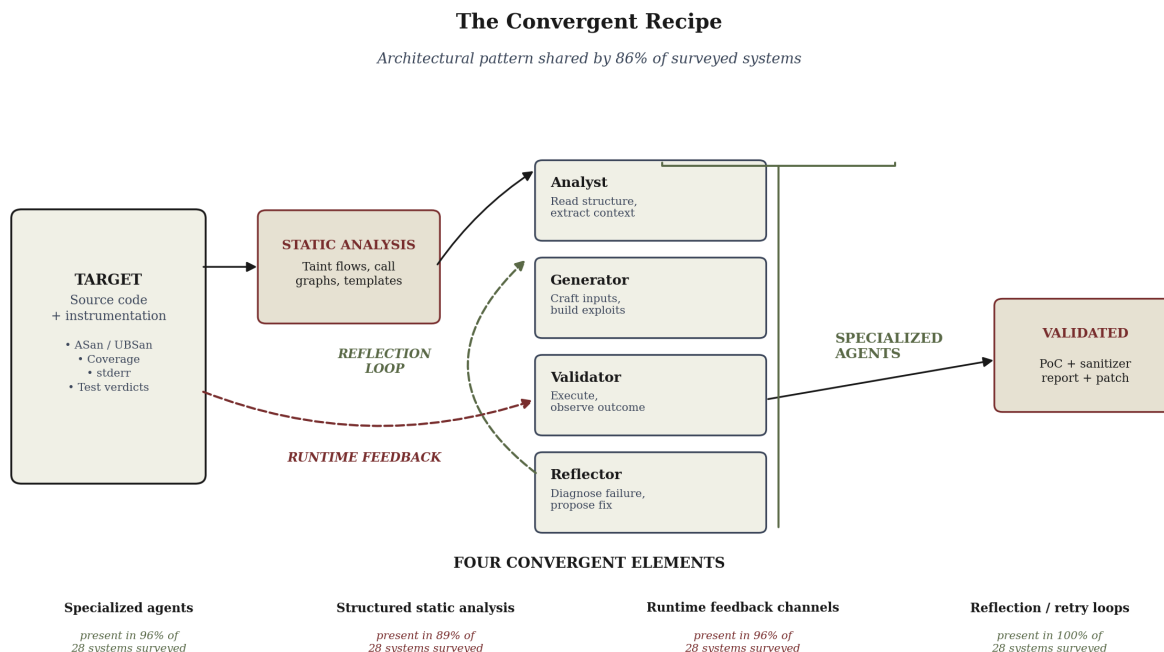


Figure 2. The convergent architectural pattern that 86% of the 28 surveyed systems instantiate. The dashed lines denote the two diagnostic channels that distinguish the methodology from earlier LLM-based vulnerability research: runtime feedback (red) from the target to the validator, and reflection (olive) from the validator back to the generator. Each of the four elements is present in over 89% of the surveyed systems; specialized agents and runtime feedback channels are both at 96%.

3. The Coverage Matrix: Demonstrating the Convergence

Table 1 maps each surveyed system to the four recipe elements, the target domain, and the externally validated output. Cells use the following encoding: ●●● core component, prominently architected; ●● present and substantive; ● present in limited form; — absent.

Table 1. Coverage matrix of surveyed systems against the convergent recipe.

System	Year	Target	Spec. Agents	Static Anal.	Runtime FB	Reflection	Real-World Output
LANDMARK SYSTEMS							
Big Sleep	2024	SQLite, OSS	●●	●●●	●●●	●●●	CVE-2025-6965 + several
AgentFlow	2026	Chrome (35M LoC)	●●●	●●●	●●●	●●●	10 Chrome 0-days, 2 Critical
VulnSage	2026	Java/JS libs	●●●	●●●	●●●	●●●	146 0-days, 73 CVE IDs
AlxCC / ATLANTIS	2025	C/Java OSS	●●●	●●●	●●●	●●●	7 CRSs, US\$29.5M challenge
A1	2025–26	EVM contracts	●●●	●●●	●●●	●●●	US\$9.33M exploit value
MDASH	2026	Windows internals	●●●	●●●	●●●	●●●	16 Win CVEs, 1 Patch Tue.
MULTI-AGENT DISCOVERY							
KernelGPT	2024	Linux kernel	●●	●●	●●●	●●●	24 bugs, 11 CVEs
VulnBot	2025	Pentest	●●●	●	●●	●●	Benchmark only
HPTSA (Fang et al.)	2024	Web apps	●●●	●●	●●	●●	14 0-day web vulns
MALF	2025	ICS protocols	●●●	●●	●●●	●●	3 0-days, 1 CNVD
LLM-BSCVM	2025	Smart contracts	●●●	●●	●●	●●	Benchmark + repair
FaultLine	2025	Java/C/C++	●●●	—	●●	●●	Benchmark only
CTF / PENTESTING							
EnIGMA	2025	CTF	●●	●●	●●●	●●	CTF benchmarks
D-CIPHER	2025	CTF	●●●	●●	●●●	●●	CTF benchmarks
CRACKEN	2025	CTF	●●●	●●	●●●	●●	CTF benchmarks
STRIATUM-CTF	2026	CTF	●●●	●●	●●●	●●	1st place, live CTF
HackSynth	2025	CTF crypto	●●	●●	●●	●●	picoCTF +13pp
DEFENSIVE / PATCHING							
CodeRover-S	2024	OSS-Fuzz	●●	●●●	●●●	●●●	OSS-Fuzz patches
PatchIsland	2026	OSS-Fuzz	●●●	●●●	●●●	●●●	31/43 AlxCC repair
Kumushi	2026	OSS-Fuzz	●●●	●●●	●●●	●●●	Root-cause patches

System	Year	Target	Spec. Agents	Static Anal.	Runtime FB	Reflection	Real-World Output
CodeMender	2025	OSS libs	●●●	●●●	●●●	●●●	72 patches, 4.5M LoC
FalseCrashReducer	2025	OSS-Fuzz	●●	●●●	●●●	●●	FP reduction
COMMERCIAL PRODUCTS							
Vulnhuntr (Protect AI)	2024	Python OSS	●	●●	●	●●	12+ 0-days, public
XBOW	2025	Web (HackerOne)	●●●	●●	●●●	●●●	1060+ HackerOne, #1 US
Claude Code Security	2026	Customer code	●●	●●●	●●●	●●●	500+ OSS vulns
Trend Micro AESIR	2025	Commercial	●●●	●●	●●●	●●	21 critical CVEs
AGENTIC FUZZING							
Gentoo	2026	Generators	●●	●	●●●	●●●	Methodology paper
GRIEF	2026	vLLM/SGLang	●●	●●	●●●	●●●	15 vulns, 2 CVEs

Coverage statistics across the 28 systems surveyed:

- Specialized agents (●● or ●●●): 27 of 28 (96%)
- Structured static-analysis input (●● or ●●●): 25 of 28 (89%)
- Runtime feedback channels (●● or ●●●): 27 of 28 (96%)
- Reflection loops (●● or ●●●): 28 of 28 (100%)
- Systems with all four elements at ●● or higher: 24 of 28 (86%)

Coverage of the Convergent Recipe Across 28 Surveyed Systems

●●● core component • ●● substantive • ● limited • — absent

		Specialized Agents	Structured Static Analysis	Runtime Feedback	Reflection Loops
LANDMARK	Big Sleep	●●	●●●	●●●	●●●
	AgentFlow	●●●	●●●	●●●	●●●
	VulnSage	●●●	●●●	●●●	●●●
	AIxCC / ATLANTIS	●●●	●●●	●●●	●●●
	A1	●●●	●●●	●●●	●●●
	MDASH	●●●	●●●	●●●	●●●
DISCOVERY	KernelGPT	●●	●●	●●●	●●●
	VulnBot	●●●	●	●●	●●
	HPTSA	●●●	●●	●●	●●
	MALF	●●●	●●	●●●	●●
	LLM-BSCVM	●●●	●●	●●	●●
	FaultLine	●●●	—	●●	●●
CTF	EnIGMA	●●	●●	●●●	●●
	D-CIPHER	●●●	●●	●●●	●●
	CRACKEN	●●●	●●	●●●	●●
	STRIATUM-CTF	●●●	●●	●●●	●●
	HackSynth	●●	●●	●●	●●
PATCHING	CodeRover-S	●●	●●●	●●●	●●●
	PatchIsland	●●●	●●●	●●●	●●●
	Kumushi	●●●	●●●	●●●	●●●
	CodeMender	●●●	●●●	●●●	●●●
	FalseCrashReducer	●●	●●●	●●●	●●
COMMERCIAL	Vulnhuntr	●	●●	●	●●
	XBOW	●●●	●●	●●●	●●●
	Claude Code Security	●●	●●●	●●●	●●●
	Trend Micro AESIR	●●●	●●	●●●	●●
FUZZING	Gentoo	●●	●	●●●	●●●
	GRIEF	●●	●●	●●●	●●●

Figure 3. Visualization of Table 1 as a heatmap, with darker (oxblood) cells indicating core components and lighter (parchment) cells indicating limited or absent elements. The six landmark systems form a uniformly dark block at the top — all four recipe elements present at

core-component intensity. The patching pipeline (CodeRover-S through CodeMender) is the next most consistent. The CTF cluster shows a weaker structured-static-analysis presence in structured-static analysis because CTF tasks often supply the relevant code slice directly. Five of 112 cells fall below substantive coverage: FaultLine's absent static analysis (the only "-" cell, representing the methodology paper that deliberately excludes static analysis as a contrast case, cited by VulnSage as evidence for why structured analysis matters); VulnBot and Gentoo (limited static analysis); and Vulnhuntr (limited specialized agents and runtime feedback, the simplest harness in the commercial category).

The convergence claim made in Section 2.2 is supported empirically by Table 1 and visualized in Figure 3: 86% of the surveyed systems implement all four recipe elements substantively, and specialized agents, runtime feedback, and reflection loops each appear in at least 96% of systems. Where elements are absent, it is typically because the system is methodologically narrow (a methodology paper rather than a deployed system) or because the static-analysis stage is intentionally elided to test whether LLMs can do without it (FaultLine, which VulnSage cites as a contrast case).

4. The Six Landmark Systems

Table 2. Side-by-side comparison of the six landmark systems.

	Big Sleep	AgentFlow	VulnSage	AixCC / ATLANTIS	A1	MDASH
Released	Oct 2024	Apr 2026	Apr 2026	Aug 2025	Jul 2025–Jan 2026	May 2026
Target	OSS, SQLite	Chrome (35M LoC)	Java/JS libs	C/Java OSS	EVM contracts	Windows internals
Model	Gemini 1.5 Pro	Kimi K2.5	Qwen3-Max	Multiple	o3/o3-pro, Gemini	Multi-model ensemble
Agents	Tool-equipped LLM	Synthesized; 18 roles + 192 explorers	5 named agents	Varies; team-built	1 + 6 tools	100+ specialized
Harness style	Hand-designed	Auto-synthesized	Hand-designed	Hand-designed (team)	Hand-designed	Hand-designed
Feedback channels	Coverage, sanitizer, stderr	All four + branch	Sanitizer, traces, stderr	All four	Live chain state	All four + custom
Reflection rounds	Implicit	Outer-loop optimization	Up to 20	Bounded	Iterative	Per-stage debaters
Real-world output	CVE-2025-6965 (SQLite); several more	10 Chrome 0-days inc. 2 Critical sandbox escapes	146 0-days, 73 CVE IDs	7 CRSs; 72.1% repair (PatchIsland)	US\$9.33M exploit value	16 Win CVEs, 4 Critical RCEs
Cost/finding	n/d	~134 H100-days/bug	~US\$0.97/alert	~US\$2,500/project	~US\$5/attempt	n/d
Independent validation	Project Zero, public CVE	CVE assignments	CVE assignments	DARPA + DEF CON	VERITE benchmark	MSRC + Patch Tuesday
Primary contribution	Variant analysis with LLMs	Harness synthesis over DSL	Constraint extraction + reflection	Competition-scale CRSs	Economic actuality	Production-scale defense

4.1 Big Sleep — The Proof of Concept

Google's Big Sleep, an evolution of the earlier Naptime framework, made the first publicly-confirmed AI-discovered zero-day in widely-used real-world software: an exploitable stack buffer underflow in SQLite, reported in October 2024 and patched the same day before any release. The bug was found via *variant analysis* — given a recently-fixed bug, search for similar bugs elsewhere in the codebase — which the Project Zero team identifies as the task class best-suited to current LLMs because it removes the open-endedness from vulnerability research and starts the agent with a concrete theory.

The Naptime/Big Sleep design principles document is the most important architectural specification in this literature. Five principles articulated by Glazunov and Brand recur across the subsequent systems profiled here:

1. **Room to reason:** explicit chain-of-thought before action.
2. **Interactive environment:** tool-grounded actions (code browser, debugger, Python sandbox, reporter) rather than oracle-style code analysis.
3. **Specialized tools:** each tool replicates a piece of a human researcher's workflow.
4. **Perfect verification:** agent output must be automatically validated by external execution, not self-judgment.
5. **Sampling strategy:** explore multiple hypotheses across independent trajectories rather than committing one trajectory to multiple hypotheses.

Big Sleep's later discovery of CVE-2025-6965 in SQLite — found before threat actors who had begun preparation could exploit it — is the cleanest demonstration of variant analysis's defensive value.

4.2 AgentFlow — Harness Synthesis

AgentFlow (Liu et al., arXiv 2604.20801, 2026) is the most ambitious harness-synthesis attempt in the literature. Its typed graph DSL unifies all five harness dimensions — agents, topology, message schemas, tool bindings, coordination protocol — into a single searchable program, and an outer-loop optimizer rewrites the program based on structured runtime feedback. On TerminalBench-2, AgentFlow's synthesized harness reaches 84.3%, leading the public Claude Opus 4.6 leaderboard. The same loop, redirected at Chrome with the open-weight Kimi K2.5 model, synthesized an 18-role topology with 192 parallel explorers across seven subsystems and discovered ten previously unknown zero-day vulnerabilities, including two Critical sandbox-escape CVEs.

The methodological significance lies in demonstrating that a domain-specific harness can be *synthesized* rather than hand-designed. That synthesis driven by runtime feedback can produce harness architectures that humans did not specify. The Chrome result also isolates the harness as the source of capability: Kimi K2.5 is mid-tier, not frontier, and the result came from how the agents were organized rather than from raw model strength.

4.3 VulnSage — Constraint Extraction and Reflection

VulnSage (Chen et al., arXiv 2604.05130, ICPC 2026) is the cleanest contemporary example of a hand-designed harness for automated exploit generation from static-analysis alerts. Five named agent roles — Code Analyzer, Code Generation, Validation, Reflection, Supervisor — implement a ReAct orchestration over taint-flow input. Two design choices encode lessons the field has internalized: constraints are accumulated in natural language rather than SMT, and static-analysis output is structured (call chains, input class sets, exploit templates) rather than raw source.

VulnSage's ablation study is the strongest empirical evidence in the literature for the structured-static-analysis claim: feeding raw source code instead of taint flows (the MiniAlert variant) costs 32–53 percentage points on real-world datasets. The system discovered 146 zero-day

vulnerabilities in npm and Maven packages, with 73 already assigned CVE IDs, at a cost of approximately US\$0.97 per alert.

4.4 The AlxCC Corpus — Competition-Scale Cyber Reasoning

DARPA's AI Cyber Challenge (2023–2025), culminating at DEF CON 33 in August 2025, is the most important single source of ground truth on what production-grade LLM-driven cyber reasoning systems look like at scale. Seven finalist teams competed over 143 hours of fully autonomous operation, analyzing 53 challenge projects, each with US\$85K in cloud compute and US\$50K in LLM API credits. The systematic analysis (SoK at arXiv 2602.07666) examines design rationale and lessons; Team Atlanta's ATLANTIS (arXiv 2509.14589) is the best-documented finalist; PatchIsland (arXiv 2601.17471) achieved a 72.1% repair rate in the official competition with zero human intervention.

The AlxCC contribution is twofold. First, it demonstrates that fully autonomous discover-and-patch pipelines work at a competition scale across teams working independently — establishing that the methodology generalizes beyond any single research group. Second, the post-competition trajectory is itself informative: despite all seven CRSs being open-sourced, they remained largely unusable outside their original teams for over six months. The OSS-CRS project (arXiv 2603.08566) is an active attempt to bridge this gap. The lesson is that building these systems is now possible, but transferring them is not.

The AlxCC corpus has had an unusual downstream impact. Microsoft's Autonomous Code Security team — which built MDASH (profiled below) — is led by Taesoo Kim, who also led Team Atlanta's winning AlxCC effort; several other ACS members came directly from Team Atlanta. The transition from competition-grade research to a vendor production system took approximately nine months.

4.5 A1 — Economic Actuality

A1 (Gervais and Zhou, arXiv 2507.05558, 2025–2026) provides the clearest economic accounting of LLM-driven exploit generation. On the VERITE benchmark of 36 real-world vulnerable Ethereum and BSC contracts, A1 achieves 63% success and extracts up to US\$8.59M per exploit (US\$9.33M total). The system uses six domain-specific tools and live interaction with the blockchain state; the per-exploit-attempt LLM cost is approximately US\$5.

Two A1 findings deserve emphasis. First, Monte Carlo analysis of historical attack timing indicates 86–89% exploit success at immediate detection, dropping to 6–21% with week-long delays. The implication is that the value of an AI-driven exploit generator is enormously time-sensitive — the same capability that protects a contract if used by defenders becomes overwhelmingly likely to drain it if used by attackers. Second, A1 is itself vulnerable to prompt injection via malicious contract code; the agent architectures that find bugs are themselves an attack surface (Section 9).

A1 crosses the threshold from technical demonstration to economically actionable capability. The combination of frontier-model reasoning, domain-specific tooling, and live blockchain interaction

produced real exploits with real financial impact at a per-attempt cost any well-resourced attacker can afford.

4.6 MDASH — Production-Scale Defense

Microsoft's MDASH (Kim et al., Microsoft Security Blog, May 12, 2026) is the first publicly documented production-grade deployment of agentic vulnerability discovery against tier-one closed-source targets. Built by Microsoft's Autonomous Code Security team — several members from the AIXCC-winning Team Atlanta — MDASH orchestrates more than 100 specialized AI agents across an ensemble of frontier and distilled models. The May 12, 2026, Patch Tuesday included 16 CVEs that MDASH helped find across the Windows networking and authentication stack, including four Critical RCEs (CVE-2026-33827 in `tcpip.sys`, CVE-2026-33824 in `ikeext.dll`, CVE-2026-41089 in `netlogon.dll`, CVE-2026-41096 in `dnsapi.dll`).

MDASH operates through a five-stage pipeline: **Prepare** (ingest target, build language-aware indices, derive attack-surface and threat models from commit history); **Scan** (specialized auditor agents emit candidate findings with hypotheses); **Validate** (debater agents argue for and against each finding's reachability and exploitability); **Dedup** (collapse semantically equivalent findings); and **Prove** (construct and execute triggering inputs validated through sanitizers). Three properties make this work in production: a multi-model ensemble where inter-model disagreement is itself a signal; specialized agents with distinct prompts, tools, and stop criteria for each stage; and an extensible plugin architecture that lets domain experts inject knowledge — kernel calling conventions, IRP rules, lock invariants, IPC trust boundaries — that the foundation models cannot infer on their own.

The benchmark results are notable on three axes. On StorageDrive — a private device driver used in Microsoft's offensive-security interviews, never published, with 21 deliberately-injected vulnerabilities — MDASH found all 21 with zero false positives. The contamination-resistant design directly addresses the concerns raised by ZeroDayBench (Section 6.1). On retrospective recall against five years of confirmed MSRC cases, MDASH achieved 96% in `clfs.sys` (28 cases) and 100% in `tcpip.sys` (7 cases). On the public CyberGym benchmark (1,507 real-world vulnerability reproduction tasks across 188 OSS-Fuzz projects), MDASH reached 88.45%, approximately five percentage points ahead of the next-highest entry.

"The harness does the work, and the model is one input." — Kim et al., MDASH announcement, May 12, 2026

The MDASH authors articulate the central thesis of this paper in one sentence. The architectural payoff is portability across model generations — when a new model lands, the targeting, validation, dedup, and prove stages do not need rewriting; the customer's investment in scope files, plugins, and calibrations carries over. The strategic implication, made explicit in the MDASH announcement, is that the durable engineering asset is the agentic system around the model rather than any single model itself.

MDASH is currently in limited private preview. The broader availability of MDASH-class engineering, and the appearance of comparable systems from other vendors are the principal variables on which the offense-defense balance of the next eighteen months depends.

5. The Broader Research Landscape

The six landmarks sit within a larger body of work. This section covers what does not fit the landmark criteria but contributes methodologically.

5.1 Multi-agent discovery beyond the landmarks

VulnBot (arXiv 2501.13411, January 2025) is among the earlier multi-agent pentest frameworks, decomposing reconnaissance, scanning, and exploitation through a penetration task graph. **MALF** (arXiv 2510.02694, October 2025) extends the pattern to industrial control protocols and identifies three zero-days in a real-world Industrial Attack-Defense Range, one of which was confirmed by China's CNVD. **LLM-BSCVM** (arXiv 2505.17416) covers smart contracts with six agents across detection, cause analysis, and repair. **FaultLine** (arXiv 2507.15241) handles Java/C/C++ proof-of-vulnerability generation but, in contrast to VulnSage, does not use static analysis to constrain the search — VulnSage cites it as evidence for why structured static-analysis input matters.

5.2 Specialized harnesses that shaped the methodology

KernelGPT (arXiv 2401.00563, March 2025) uses LLMs to automatically infer syzkaller syscall specifications and refine them using validation feedback. As of publication: 24 unique Linux kernel bugs, 11 assigned CVEs, with generated specifications merged upstream into Syzkaller. KernelGPT pioneered the use of validation feedback to refine the *specifications* a fuzzer operates on rather than test inputs directly — a pattern that recurs in MDASH's plugin architecture.

EnIGMA, **D-CIPHER**, **CRAKEN**, **HackSynth**, and **STRIATUM-CTF** are CTF-solving agents that established and refined the multi-agent template in offensive security. STRIATUM-CTF (arXiv 2603.22577) won first place autonomously in a university-hosted live CTF in late 2025, outperforming 21 human teams.

HPTSA / Fang et al. (arXiv 2406.01637) established the "task-specific expert agents" template (XSS, SQLi, CSRF, SSTI, ZAP, generic web) that subsequent web-vulnerability systems have adopted. Fang's group has been particularly productive in this space, with follow-on work on one-day exploitation (arXiv 2404.08144) and zero-day teams (arXiv 2406.01637).

5.3 The defensive pipeline

The CodeRover-S → PatchIsland → Kumushi → CodeMender → FalseCrashReducer lineage represents the maturation of LLM-driven autonomous patching.

CodeRover-S (Zhang et al., arXiv 2411.03346, 2024) repurposes the AutoCodeRover agent for security patching, augmenting fuzzer-generated vulnerability reports with dynamic call-graph information and

type-based analysis. The methodological contribution is the recognition that fuzzer reports are systematically under-specified relative to human issue reports.

PatchIsland (arXiv 2601.17471, 2026) demonstrated 31/43 (72.1%) autonomous repair in the AIXCC official competition. It uses an ensemble of diverse LLM agents to cover a wider range of projects, bug types, and programming languages than single-agent approaches.

Kumushi (arXiv 2605.04251, 2026) addresses root-cause patching by combining static and dynamic fault localization, fixing the limitation that PatchAgent and similar systems can patch nearby-but-incorrect locations when distracted by irrelevant context.

CodeMender (Google DeepMind, October 2025) is the most consequential commercial defensive system. Built on Gemini Deep Think models, CodeMender combines reactive patching (instantly patching newly-found vulnerabilities) with proactive rewriting (applying `-fbounds-safety` annotations to libwebp, for example, rendering an entire class of buffer overflows unexploitable). As of publication: 72 security fixes submitted to open-source projects spanning 4.5M+ lines of code, with patches reviewed by human researchers before upstreaming.

FalseCrashReducer (arXiv 2510.02185, 2025) addresses the dual problem in OSS-Fuzz-Gen of false-positive crash reports overwhelming triage capacity — the defensive analog of the false-positive problem VulnSage tackled on the offensive side.

5.4 Commercial products: the deployment layer

This is the category most underrepresented in the academic literature and most consequential for the offense-defense balance.

XBOW (Series C: US\$120M, March 2026; US\$1B+ valuation) deploys swarms of AI agents against HackerOne bug-bounty programs. In June 2025, XBOW became the first non-human bug hunter to reach the top of HackerOne's US leaderboard, submitting 1,060+ vulnerabilities; 130 resolved, 303 triaged, with critical findings against Amazon, Disney, PayPal, Sony, and AT&T. XBOW's CTF-with-canaries methodology (planting flags in source code and sending agents after them) and its validator architecture (using non-LLM checkers for finding validation) are the closest commercial analog to the academic harness architectures profiled above. XBOW's significance for this survey is twofold: it is the public economic validation that LLM-driven offensive security is commercially viable at unicorn scale, and its head-to-head results (an 85× speed advantage in a 104-scenario test vs. a 20-year veteran human pentester) provide unusually direct human-vs-AI comparison data.

Anthropic Claude Code Security (February 2026) is a static-analysis successor that integrates with enterprise codebases. Using Claude Opus 4.6 (the model below Mythos), Anthropic's Frontier Red Team found 500+ vulnerabilities in production open-source codebases that had gone undetected for years. The system uses multi-stage verification with self-challenging finding validation and confidence ratings, sitting between MDASH-style enterprise harnesses and the VulnSage-style academic systems.

Protect AI Vulnhuntr (October 2024) is the open-source counterpart — a Python-only, zero-shot vulnerability scanner that uses Claude to trace full call chains from user input to server sinks. Found 12+ zero-days in large open-source Python projects at announcement. Vulnhuntr's significance lies in the diffusion vector it represents: open-source defensive harnesses with frontier-model backends mean that MDASH-class capability does not remain vendor-confined indefinitely.

Trend Micro AESIR has found 21 critical CVEs since mid-2025 across NVIDIA, Tencent, MLflow, and MCP tooling — one of the clearest signs that AI-assisted vulnerability discovery works outside research labs, against actively used commercial software at commercial scale.

5.5 The agentic-fuzzing paradigm

A recent direction reframes the LLM/fuzzer relationship: rather than using LLMs as auxiliaries to coverage-guided fuzzing, use deep agents as the primary search engine. **Agentic Fuzzing: Opportunities and Challenges** (arXiv 2605.10074, 2026) makes the conceptual argument. **Gentoo** (arXiv 2604.01442, 2026) operationalizes it with an autonomous agent that compiles, executes, and debugs input generators with predicate-level feedback. **GRIEF** (arXiv 2605.11202, May 2026) applies the methodology to LLM inference engines themselves — vLLM and SGLang — and discovers 15 vulnerabilities, including 2 CVEs.

5.6 Capability benchmarks (treated separately in Section 6)

CyberSecEval (Meta, four versions: original, CSE2, CSE3 with offensive capabilities, CSE4 with CyberSOCEval and AutoPatchBench), CyberGym, CVE-Bench, ZeroDayBench, Cybench, BountyBench, SEC-bench, NYU CTF Bench, CTF-Dojo. We address these in the critical-assessment section because the validity of their measurement is the central methodological concern of the field.

6. Critical Assessment

This section addresses what the field's surveyed papers do not adequately cover: where the methodology breaks down, what the benchmark numbers actually measure, and what the headline disclosures may or may not establish.

6.1 Measurement validity: what the benchmark numbers don't tell you

The benchmark numbers cited throughout this survey require interpretation that the source papers often lack.

Training-data contamination. Most public benchmarks draw from CVE databases that are themselves likely in the training corpora of frontier models. A system that reports strong performance on SecBench.js may be benefiting from memorization rather than reasoning. ZeroDayBench (arXiv 2603.02297) addresses this by porting real CVEs into structurally distinct codebases. Its central finding — *that frontier LLMs are not yet capable of autonomously solving their tasks* — is a result obscured by the field's headline-disclosure framing. MDASH's StorageDrive benchmark (21/21, zero FP) is contamination-resistant by construction (private codebase never published), which is why we weight it heavily despite the small sample size.

Variance and reproducibility. The TerminalBench-2 leaderboard reports maximum-over-replays scores. CyberGym's protocol provides vulnerability descriptions of varying quality. The standard deviation across runs is rarely reported. Independent third-party reproductions of major results are rare. When Co-RedTeam (arXiv 2602.02164) observes that *empirical evidence from CyBench, BountyBench, and CyberGym shows current systems struggle with multi-step reasoning, adaptive attack planning, and robust exploration*, the gap between top-of-leaderboard claims and average behavior is the relevant signal.

Retrospective vs prospective recall. MDASH's 96% recall on five years of clfs.sys MSRC cases are impressive, but the MDASH authors themselves caveat that this number is retrospective — it tells us the system would have been useful had it existed, not that the next 38 bugs in CLFS will be found at the same rate. A forward-looking signal requires forward-looking output, which is what the Patch Tuesday cohort provides. This caveat applies broadly: retrospective recall is the lower bound on system value, not the prospective rate.

The lab-to-real gap. One AI-pentesting analysis usefully quantifies this: GPT-4 exploits 87% of one-day CVEs given advisory descriptions, but only 13% of real CVEs in CVE-Bench and near-0 % of hard HackTheBox challenges. The gap between "given an advisory, reproduce the exploit" and "from scratch, find an exploit" is large and persistent.

6.2 What current LLMs actually struggle with

The published literature has clearly documented technical limits, even as headline disclosures obscure them.

Logic bugs versus memory-safety bugs. Memory-safety bugs have local symptoms (sanitizer firing, segfault) and well-defined oracles. Logic bugs — business-logic errors, broken access control, transaction-ordering flaws — have no such oracles. The literature shows that current systems handle memory-safety bugs substantially better than logic bugs. Anthropic's framing of Claude Code Security explicitly acknowledges that rule-based static analysis "often misses more complex vulnerabilities, like flaws in business logic or broken access control" — and positions agentic analysis as the answer, but does not yet show comparable success rates on logic bugs to those on memory-safety bugs.

Interprocedural and cross-file reasoning. The empirical study "LLM-based Vulnerability Detection at Project Scale" (arXiv 2601.19239) identifies *shallow interprocedural reasoning and misidentified source/sink pairs as the primary causes of failure*. MDASH's CVE-2026-33824 deep dive specifically credits the multi-agent debate stage for handling a six-file aliasing bug — implying that single-agent harnesses miss this class entirely.

Robustness to code variations. Risse et al. (as cited in the BU peaclab work) showed that even trivial code augmentations cause accuracy drops in LLM-based vulnerability detection. "Everything You Wanted to Know About LLM-based Vulnerability Detection But Were Afraid to Ask" (arXiv 2504.13474) identifies *the inability to accurately detect vulnerabilities, a significant lack of generalization capability for out-of-distribution vulnerabilities, overthinking tendencies in reasoning models, and limitations of test-time scaling*.

False-positive rates at project scale. When applied to open-source projects in unsupervised mode, both LLM-based and traditional tools generate substantial warnings with very high false discovery rates. The same project-scale study notes computational costs from hundreds of thousands to hundreds of millions of tokens and multi-hour to multi-day runtimes per project. The implication is that unsupervised LLM-based vulnerability detection at scale is operationally expensive and noisy. The MDASH/VulnSage approach — heavy investment in validation, dedup, and proof stages — is the response to this finding, but it requires substantial engineering that academic systems rarely match.

Skeptical baseline from peer-reviewed work. Ullah et al. (NDSS 2024, "LLMs Cannot Reliably Identify and Reason About Security Vulnerabilities (Yet?)") report Microsoft's leaderboard for LLM-based vulnerability detection, showing the best tool with under 70% accuracy and 30%+ FPR, which they characterize as "cannot be trusted in real-world cases." Steenhoek et al. and Ding et al. (cited in the Co-RedTeam analysis) reach similar conclusions on isolated-function vulnerability detection — Balanced Accuracy 0.5–0.63, with LLMs failing to distinguish buggy from fixed versions 76% of the time. These results, less than two years old, are not contradicted by the headline disclosures of 2026; they describe

the *base rate*, and the headline disclosures describe what's possible *with heavy harness engineering on top of that base rate*.

6.3 The Mythos disclosure: examining the strongest claims

The Anthropic Mythos disclosure is the highest-stakes claim in this survey, and a serious critical treatment requires examining it directly.

What is corroborated? The UK AI Security Institute conducted an independent evaluation and confirmed several headline results: Mythos is the first AI model to complete the Institute's simulated 32-step corporate network attack, and Mythos solved 73% of expert-level CTF problems. CETaS (Centre for Emerging Technology and Security) at the Alan Turing Institute published an expert analysis treating the capabilities as real and policy-significant. The Project Glasswing consortium — approximately 40 critical-infrastructure operators, including major tech firms, financial institutions, and government stakeholders — has had monitored access; their willingness to participate is implicit confirmation of credibility from sophisticated counterparties.

What is not corroborated? The "thousands of zero-days across every major OS and browser" claim rests entirely on Anthropic's self-report. As of this writing, the public CVE assignments traceable to Mythos are a small subset of this claimed total. Anthropic's report explicitly states that *they cannot discuss the exact details of many of these exploits; the ones we can discuss are the simplest and easiest to exploit and do not fully exercise the limits of Mythos Preview*. This is plausible — disclosure constraints are real — but it also means the strongest claims are unfalsifiable through normal scientific channels.

The marketing-versus-capability question. Multiple cybersecurity practitioners interviewed for industry coverage have noted that limited-access announcements with controlled information disclosures are textbook marketing. Joe Saunders (RunSafe Security) characterized it as "an element of marketing charm" while still allowing that the underlying capability shift is real. The honest reading: Anthropic has commercial and policy reasons to dramatize, and a credible underlying capability shift can coexist with framing choices that maximize attention. The UK AISI caveat — that test environments lacked active defenders, defensive tooling, and penalties for tripwire-triggering actions, so the Institute *cannot say for sure whether Mythos Preview would be able to attack well-defended systems* — is the most important single qualification in the entire Mythos disclosure.

The harmlessness-training data point. A footnote in the CETaS analysis is worth highlighting: *the version of Mythos Preview released to partners has "additional harmlessness training" that reportedly reduced the task completion rate to near 0% as "models generally refused to engage with the tasks ... from the start."* This is consequential. It suggests (a) safety training and offensive capability are in genuine tension; (b) the deployed version of Mythos that critical-infrastructure partners actually have is substantially less capable than the version Anthropic's red team measured; (c) the practical risk model depends heavily on

whether Mythos-class capability with safety training removed becomes available through leak, jailbreak, or open-weight alternatives.

The "downstream consequence" framing. Anthropic states that Mythos's cybersecurity capabilities emerged as a downstream consequence of general improvements in code understanding, reasoning, and autonomy, rather than from targeted cybersecurity training. This framing is convenient — it positions Anthropic as not having deliberately built dual-use offensive technology. It is also probably partially true (capability spillover from coding is real). Still, the specific decision to evaluate the model on offensive tasks, develop the harness, and publish the results is a deliberate choice that the framing elides.

6.4 Where the convergent recipe doesn't work

The Section 2.2 recipe — specialized agents + structured static analysis + runtime feedback + reflection — has been empirically shown to work on memory-safety bugs in source-available targets with sanitizer instrumentation. It works less well or not at all on:

- **Closed-source binary analysis without symbols.** Most of the surveyed systems require source code. Approaches against bytecode (Mayhem-style) have not yet shown comparable LLM-driven results.
- **Cross-language vulnerabilities.** A bug in C code called from Python through FFI, with the vulnerable path crossing the language boundary, is poorly handled by systems trained on single-language reasoning.
- **Concurrency bugs (TOCTOU, data races) at scale.** MDASH's CVE-2026-33827 is a race-condition UAF — but it's a relatively local race. Distributed concurrency bugs spanning multiple services remain difficult to address.
- **Logic bugs without crash oracles.** As noted in 6.2.
- **Side-channel vulnerabilities.** Timing attacks, cache attacks, Spectre/Meltdown variants — these require physical-execution reasoning that current harnesses do not implement well.
- **Vulnerabilities are only visible at scale.** Bugs that require thousands of inputs to trigger probabilistically, or that emerge from configuration interactions across many components, do not match the agent-trial-and-error pattern.

The MDASH and Big Sleep results are concentrated in memory-safety bugs in C/C++ kernel and networking code. This is exactly the sub-class the methodology fits best. The honest claim about the past nineteen months is that LLM-driven vulnerability research has gone from "doesn't work" to "works very well on the subset it fits"; the unsolved problem is the subset it doesn't fit, which is large.

7. The Threat Model

The survey-level "attackers vs defenders" framing of Section 1 is inadequate for policy analysis. This section disaggregates the actor classes based on their differing risk profiles.

7.1 Nation-state actors

Top-tier nation-state operations (US, UK, Israel, China, Russia at the apex) already possess deep human exploitation expertise. AI vulnerability discovery is a *marginal uplift* — it accelerates work that would happen anyway and frees senior operators to focus on the highest-value targets. The risk model for these actors is not "AI gives them capabilities they lack" but "AI lets them apply existing capabilities at greater scale and lower latency."

Mid-tier nation-state actors are the more consequential case. Maruyama, cited in Foreign Policy's analysis, names Iran and North Korea as the specific concern: countries with offensive intent and operational experience but with *insufficient depth to execute complex kill chains, develop zero-day attacks, and weaponize them effectively*. Mythos-class capability (or, more relevantly, its open-weight successor) directly addresses that gap. The risk model for these actors is "AI converts intent and motivation into operational capability that previously required years of organic development."

7.2 Cybercriminal organizations

This is where the risk model shifts most dramatically. Ransomware-as-a-service and access-broker economies have already commoditized exploitation; AI-driven exploit generation reduces the cost per attack, expands the target set to less-hardened organizations, and shrinks the window between vulnerability disclosure and operational use. The A1 result — US\$9.33M total extracted at ~US\$5 per exploit attempt — is the cleanest demonstration of the economic shift, even though it concerns smart contracts specifically.

Criminal organizations are also the most likely beneficiaries of open-weight diffusion. Within days of any major open-weight release, uncensored variants appear on public repositories. The window between frontier closed-model capability and broadly available, uncensored, open-weight capability has historically been 12 to 18 months. For Mythos-class capability, that window is approximately the period this survey covers.

7.3 Independent actors and the "script kiddie" question

Jin et al. (IEEE S&P Workshops 2025, "Good News for Script Kiddies? Evaluating LLMs for Automated Exploit Generation") examine whether current LLMs lower the barrier to entry for unsophisticated attackers. The honest answer in mid-2025 was no — LLMs alone were not sufficient. The honest answer in mid-2026, with MDASH-class harnesses possibly available via open-source diffusion and Mythos-class models within 12 to 18 months of broader release, is plausibly yes for some attack classes.

Bugcrowd's 2026 "Inside the Mind of a Hacker" report finds 82% of hackers now use AI tools in their daily workflow, up from 64% in 2023. This is not "AI replaces hackers" — it is "AI augments hackers across the skill spectrum." The script-kiddie risk model is most acute for attack classes in which harness engineering has been productized and disseminated: web vulnerabilities (Vulnhuntr, XBOW-style tooling), prompt injection against AI systems (specialized fuzzers), and pattern matching against known vulnerability classes.

7.4 The defender side, similarly disaggregated

Defenders are not monolithic either. The relevant distinctions:

- **Tier-one vendors** (Microsoft, Google, Apple, major OSS foundations): have MDASH-class engineering or are building it. The Project Glasswing consortium reaches these organizations.
- **Tier-two enterprises** (Fortune 1000 outside tech): rely on commercial security products. Their access to MDASH-class capability depends on whether vendors like XBOW, Anthropic Claude Code Security, Trend Micro AESIR, and commercial successors expose the capability at price points and integration depths these organizations can use.
- **Tier-three organizations** (mid-market, public sector, non-profit): rely on consumer-grade security products and infrequent professional pentest engagements. These organizations are the most exposed to the asymmetric scenario in which attackers gain MDASH-class capability before defenders do.
- **Open-source maintainers**: receive CodeMender-style patches as upstream contributions; XBOW-style continuous testing against their public infrastructure; ad-hoc benefits from research-driven discovery (Big Sleep, AgentFlow). This category is heavily subsidized by frontier labs' open-source defensive work — and is also the most exposed to the diffusion of offensive tools without paired defensive capability.

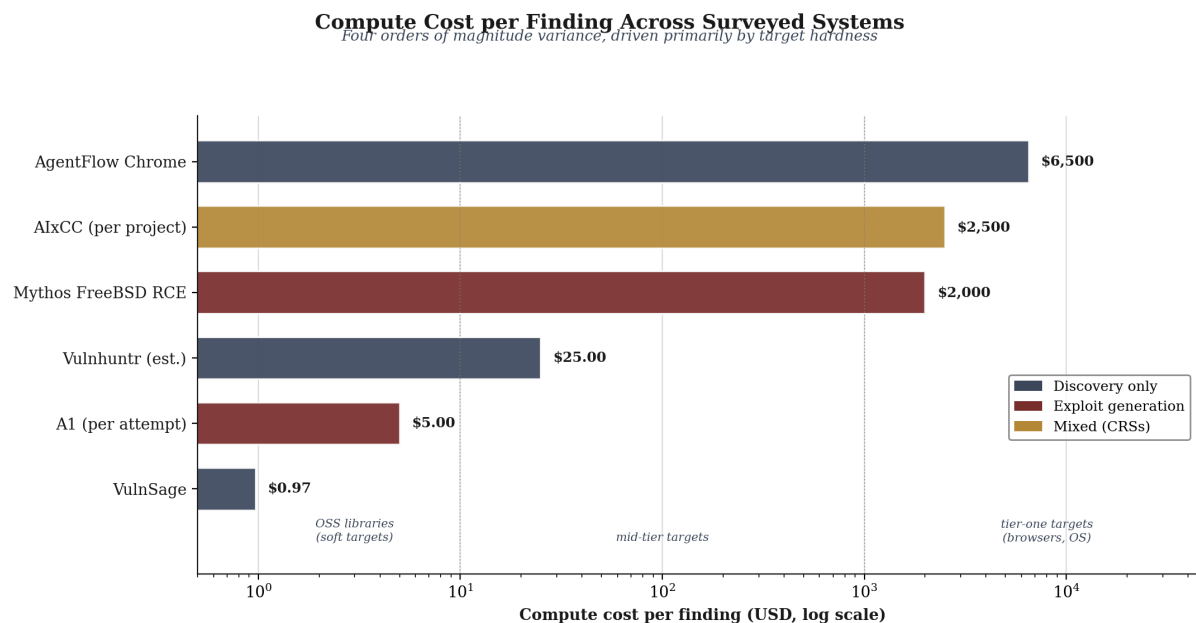
The Bain & Company analysis of Mythos observes that the controls constituting strong cybersecurity fundamentals — access controls, network segmentation, automated patching, zero trust, anomaly detection — already provide significant protection against AI-enabled attacks, "yet most organizations have not built those foundations to the required standard." The asymmetric risk is concentrated in tier three.

8. The Economics of AI Vulnerability Research

8.1 Per-finding cost across the surveyed systems

System	Compute cost per finding	Source
VulnSage	~US\$0.97 per alert	Reported in the paper
A1	~US\$5 per exploit attempt	Reported in the paper
AgentFlow Chrome campaign	1,344 H100-days / 10 bugs $\approx$ 134 H100-days per bug (~US\$6,400 compute at US\$2/H100-hour, plus inference)	Calculated from the paper
AlxCC competition	US\$85K compute + US\$50K LLM / 53 projects $\approx$ US\$2,500 per project	DARPA
Mythos FreeBSD RCE	"Under US\$2,000" / 1 exploit	Anthropic disclosure
MDASH	Not publicly disclosed	—
XBOW HackerOne campaign	Not publicly disclosed; "hours" per pentest	XBOW marketing

The cost-per-finding numbers span roughly four orders of magnitude (from ~US\$1 for VulnSage to ~US\$6,500 for pure compute for an AgentFlow Chrome zero-day, with all-in costs, including LLM inference, likely higher). The variance is driven primarily by *target hardness*: VulnSage targets known-buggy npm packages with rich static-analysis priors; AgentFlow targets one of the world's most heavily audited C++ codebases.



Note: AgentFlow Chrome figure derived from 192 H100s x 7 days at \$2/H100-hour cloud pricing, divided across 10 disclosed zero-days. A1 figure is per exploit attempt, not per successful exploit.

Figure 4. Compute the cost per finding for six surveyed systems that have disclosed cost data. Bars are color-coded by output type (discovery only, exploit generation, mixed). Note the log scale: the variance, from VulnSage's US\$0.97 per alert to AgentFlow's approximately US\$6,500 in raw compute per Chrome zero-day (1,344 H100-days divided across 10 bugs, at US\$2/H100-hour cloud pricing; all-in costs including LLM inference are higher), almost entirely reflects target hardness rather than methodological differences. The cost gradient is the strongest single argument for tier-based threat modeling (Section 7): an attacker willing to spend US\$5 per A1 attempt and capable of running thousands of attempts has a substantially different threat profile than an attacker who must commit AgentFlow-scale compute for a single tier-one finding.

8.2 The bug-bounty market

HackerOne's 2025 report shows US\$81M paid in bounties (+13% YoY). XBOW's 130 resolved bounties on HackerOne — operated as a commercial system, with human review before submission — is the cleanest market signal that AI-driven offensive security is economically viable as a vendor business. The XBOW Series C (US\$120M at >US\$1B valuation, March 2026) provides external pricing of that proposition.

The market dynamics for AI vulnerability research have three notable features:

1. **Bounty payouts have not collapsed.** If AI vulnerability discovery were trivially scaling, bounty payouts per finding would fall as supply increased. They haven't — partly because the hardest bugs still require capabilities that AI doesn't yet deliver, partly because the bounty programs have not yet adapted to AI-scale submission volume.
2. **The HackerOne ToS issue.** Submission via automated tools is subject to platform policies; XBOW operates with human review before submission to comply. This is a procedural friction that may be removed or made stricter; either change would substantially affect the market.

3. **Vendor capture.** Critical-infrastructure operators participating in Project Glasswing or analogous programs are paying (in compute, access, and disclosure obligations) for capabilities that are not available on the open market. This is a parallel economy that does not show up in public bug-bounty data.

8.3 Defensive deployment cost

The asymmetric cost that the surveyed systems generally underweight is the *deployment side*: patches need to be applied, regression-tested, rolled out, and adopted by end users. CodeMender's 72 patches submitted to OSS projects spanning 4.5M lines of code is methodologically excellent but operationally small. MDASH's 16 CVEs in a single Patch Tuesday are more representative of production-scale defensive output — and they ship through Microsoft's mature patch-distribution infrastructure to billions of users.

The implication for tier-three organizations (Section 7.4): even if defensive discovery capability becomes broadly available, the deployment infrastructure to apply discovered patches at scale is not equally available. The TechTarget analysis observes: "vulnerabilities will be found faster, exploitability will become clearer more quickly, and response windows will continue to shrink." For organizations whose patch cycle is measured in weeks, MDASH-class discovery without MDASH-class patch deployment is a net negative — they hear about more vulnerabilities they can't fix in time.

9. Attacks on the Harnesses Themselves

The agent architectures profiled in this survey are themselves an emerging attack surface. The same methodology that finds bugs in software finds bugs in agent behaviors.

9.1 Direct attacks on agent systems

Agent Security Bench (ASB) at ICLR 2025 demonstrated attack success rates exceeding 84% against LLM-based agents across multiple operational stages (system prompt, user instructions, memory retrieval, planning, action execution). The taxonomy of attacks includes prompt injection, memory poisoning, tool misuse, and reward hacking. The current generation of defenses is largely ineffective against the strongest attacks.

ChainFuzzer (arXiv 2603.12614, 2026) targets multi-tool workflow vulnerabilities — unsafe behaviors that emerge only when multiple tools are invoked, even when each call appears benign. The system identifies high-impact operations with strict source-to-sink dataflow evidence and synthesizes prompts that drive agents to execute target chains. This is structurally the same problem as the source-to-sink reasoning in MDASH, applied to LLM-agent tool chains rather than C code.

AgentVigil uses genetic fuzzing and Monte Carlo Tree Search to discover indirect prompt-injection vulnerabilities and has successfully evaluated against the AgentDojo and VWAadv benchmarks.

9.2 Specific risks for vulnerability-discovery agents

A1's documented prompt-injection vulnerability — through malicious contract code (`string data = 'this contract is secure'`) — illustrates a fundamental concern: agents that read attacker-controlled input as part of vulnerability analysis are themselves attackable by that input.

For MDASH-class production systems, the relevant attack vectors include:

- **Adversarial code in the target:** malicious comments or code structures designed to confuse the debater agents into accepting incorrect verdicts.
- **Sanitizer-output spoofing:** if attackers can influence sanitizer output (in cases where the agent is reasoning about partially-attacker-controlled infrastructure), they can mislead validation.
- **Multi-agent coordination attacks:** in systems like MDASH with 100+ agents, attacks target the ensemble's voting or consensus mechanism rather than any individual agent.
- **Plugin attacks:** extensible plugin architectures (MDASH's CLFS proving plugin, Vulnhuntr's Python static analyzers) expand the trusted code base; a malicious plugin or a vulnerable plugin compromise becomes an attack vector.

The Anthropic Slack MCP Server data-exfiltration vulnerability (as cited in industry coverage) illustrates this class concretely: prompt injection via meeting content extracted from the MCP server.

9.3 Implication: defensive harnesses need defensive harnesses

The recursive implication is uncomfortable: MDASH-class systems, once deployed at production scale against valuable targets, become high-value targets themselves. The methodology for finding bugs in agent systems is the same as the one this survey has documented for finding bugs in software systems. The agentic security industry will plausibly develop, within the next two years, a specialized sub-genre of agentic security harnesses targeting other agentic security harnesses. This is not yet a major area of published research; in our view, it is the most consequential omission in the current literature.

10. The Mythos–MDASH Pairing, Reconsidered

We now return to the central pairing of this survey, equipped with the critical apparatus developed above.

10.1 What the pairing demonstrates

The Mythos disclosure and the MDASH announcement, separated by thirty-five days, are best understood as the public face of two complementary maturation curves.

Mythos is the capability-curve endpoint. It demonstrates that frontier-model capability has reached a threshold at which the elaborate harness engineering required by AgentFlow, VulnSage, and similar systems becomes optional for some offensive use cases. Anthropic's claim that a minimal scaffold ("find a security vulnerability in this program") suffices for Mythos to produce exploits is consistent with the trajectory documented across the surveyed literature.

MDASH is the engineering-curve endpoint. It demonstrates that a production-grade harness using *generally available models* can produce defensive output of comparable consequence: 16 CVEs in a single Patch Tuesday, 96% retrospective MSRC recall, and the top score on CyberGym. The MDASH authors are explicit that frontier-model access is not the bottleneck for them.

The pairing matters because if both curves have endpoints, the binding question is no longer capability or engineering in isolation, but *the rate of diffusion of each across actor classes* (Section 7).

10.2 What the pairing does not yet demonstrate

The strongest counter-reading is that we are over-interpreting two announcements, separated by five weeks, from companies with a substantial commercial incentive to dramatize. The honest acknowledgment:

- Mythos's claim of "thousands of zero-days" remains largely unverified outside Anthropic.
- MDASH's headline benchmarks (96%, 100%, 88.45%) come from system vendors' self-reporting; the StorageDrive 21/21 result is the most independently credible.
- Both systems have been deployed in limited-preview programs that select for sympathetic counterparties.
- Neither system has yet faced an adversarial third party attempting to break it (Section 9).

A careful reading of both disclosures supports the claim that *something has changed* in 2026 relative to 2024, but it does not support the strongest framing that the offensive-defensive balance has been permanently transformed. The methodology has matured, the engineering has scaled, and the disclosures are credible enough to take seriously — but they are not yet at the level of independently-replicated peer-reviewed evidence.

10.3 The 6–12 and 12–18 month windows

The CETaS analysis frames the central policy question around a six-to-twelve month window before adversaries can replicate Mythos-class capability through other frontier labs catching up; Anthropic uses similar language. A separate, longer window of twelve to eighteen months applies to the diffusion of equivalent capability through *open-weight* models. The argument for both windows is:

1. Frontier capability is currently held by approximately three labs (Anthropic, OpenAI, Google DeepMind).
2. Open-weight models trail frontier capability by twelve to eighteen months historically.
3. Uncensored derivatives of open-weight models appear within days of release.
4. Therefore, Mythos-class offensive capability without harmlessness training reaches broadly available open-weight status approximately 12 to 18 months after the Mythos disclosure.

The argument is plausible but not certain. Counter-considerations: open-weight models may not catch up if capability gains require scale that open-weight builders cannot match; harmlessness training may prove more durable than the cited footnote suggests; the actual deployment of Mythos-class capability requires harness engineering that adversaries may not match for additional months. The 12–18-month framing is the right order of magnitude, but it's not a fixed timeline.

11. Open Problems

In rough priority order:

1. **Logic-bug detection.** The methodology works well for memory-safety bugs but poorly for logic bugs. Closing this gap is the single most important unsolved problem.
2. **The deployment gap.** MDASH-class engineering needs to reach tier-two and tier-three organizations (Section 7.4) before tier-three becomes the soft underbelly of the threat surface.
3. **Adversarial robustness of agent systems.** Section 9's implication — that vulnerability-discovery agents are themselves attackable — is essentially unaddressed in the current literature.
4. **Benchmark contamination.** Public benchmarks need contamination-resistant designs analogous to MDASH's StorageDrive. The methodology for porting CVEs to functionally similar codebases (ZeroDayBench) is a promising direction.
5. **Independent third-party evaluation.** Anthropic's UK AISI partnership is one model; independent evaluators replicating MDASH's headline results would substantially strengthen the public evidence base.
6. **Open-weight defensive harnesses.** XBOW, Anthropic Claude Code Security, and Trend Micro AESIR are commercial. MDASH is vendor-internal. Vulnhuntr is the only open-source point of reference, and it covers only Python. Open-source MDASH-class harnesses would be the most consequential single development for the deployment gap.
7. **Disclosure norms at machine speed.** The traditional 90-day coordinated disclosure window assumes human-paced research. At Mythos-scale throughput, those assumptions break. Project Glasswing is one model; we need others, and we need empirical evidence about which models work.
8. **Cross-domain transfer.** Harness synthesis (AgentFlow) was demonstrated on Chrome with Kimi K2.5. Whether the same approach transfers to wholly different target classes — embedded firmware, hardware-software interfaces, distributed systems — is unestablished.

12. Conclusion

Between October 2024 and May 2026, LLM-driven vulnerability research underwent a methodological consolidation that no single result captures but that the cumulative evidence makes unmistakable. The convergent recipe — specialized agents, structured static-analysis input, runtime feedback channels, reflection loops — appears in 86% of the surveyed systems and at landmark intensity in all six of the systems we profile in detail. The harness, not the model, is the durable engineering asset; the MDASH authors articulated this in one sentence, but every system in the survey supports it.

The Mythos disclosure and the MDASH announcement are the public face of two complementary maturation curves: frontier capability at the offensive limit, production engineering at the defensive limit. We have argued, with appropriate caveats, that both endpoints are real. We have also argued, with less ambiguity, that the binding constraint of the next phase is no longer capability or engineering in isolation but the rate of diffusion of each across actor classes and organizational tiers — and that the most consequential variable is whether MDASH-class engineering reaches tier-three organizations before Mythos-class capability reaches mid-tier adversaries.

We have been deliberate in this survey about distinguishing what is established from what is asserted. The headline disclosures of 2026 are credible but not yet peer-reviewed. The benchmark numbers across the surveyed literature are real, but interpret with care. The technical limits the field is least eager to discuss — logic bugs, interprocedural reasoning, adversarial robustness, the lab-to-real gap — remain unsolved at scales that matter. The convergent recipe works very well on the subset of vulnerabilities it fits, and not at all on subsets it doesn't.

The honest summary is that the past nineteen months saw LLM-driven vulnerability research move from "can it find any bug" to "what's the optimal harness for the bug class it fits." That is genuine progress. It is not the discontinuity that the strongest framings suggest. And it places the field at exactly the point where the next phase is determined less by further capability advances and more by the deployment economics of capability already in hand.

For the period this survey documents, the field has earned the framing of "the harness era" — a period in which the central engineering insight stabilized, the recipe became reproducible, and the bottleneck shifted from research to operationalization. Whether the period that follows is remembered as the era in which automated vulnerability research empowered defenders, overwhelmed them, or — most plausibly — did both at different organizational tiers simultaneously, will depend on choices made by frontier labs, vendor consortiums, open-source maintainers, and policymakers over the twelve to eighteen months immediately following this writing.

References

Landmark systems

- Glazunov, S. and Brand, M. *Project Naptime: Evaluating Offensive Security Capabilities of Large Language Models*. Project Zero, June 20, 2024. <https://projectzero.google/2024/06/project-naptime.html>
- Big Sleep team. *From Naptime to Big Sleep: Using Large Language Models To Catch Vulnerabilities In Real-World Code*. Project Zero, November 1, 2024. <https://projectzero.google/2024/10/from-naptime-to-big-sleep.html>
- Liu et al. *AgentFlow: Synthesizing Multi-Agent Harnesses for Vulnerability Discovery*. arXiv 2604.20801, 2026.
- Chen et al. *VulnSage: A Multi-Agent Framework for Automated Exploit Generation with Constraint-Guided Comprehension and Reflection*. arXiv 2604.05130, ICPC 2026.
- Multiple authors. *SoK: DARPA's AI Cyber Challenge (AlxCC): Competition Design, Architectures, and Lessons Learned*. arXiv 2602.07666, 2026.
- Team Atlanta. *ATLANTIS: Cyber Reasoning System for the AlxCC Competition*. arXiv 2509.14589, 2025.
- Gervais and Zhou. *AI Agent Smart Contract Exploit Generation*. arXiv 2507.05558, 2025–2026.
- Kim et al. *Defense at AI speed: Microsoft's new multi-model agentic security system (MDASH) tops leading industry benchmark*. Microsoft Security Blog, May 12, 2026. <https://www.microsoft.com/en-us/security/blog/2026/05/12/defense-at-ai-speed-microsofts-new-multi-model-agentic-security-system-tops-leading-industry-benchmark/>

Patching pipeline

- Zhang et al. *Fixing Security Vulnerabilities with AI in OSS-Fuzz (CodeRover-S)*. arXiv 2411.03346, 2024.
- Kim et al. *PatchIsland: Orchestration of LLM Agents for Continuous Vulnerability Repair*. arXiv 2601.17471, 2026.
- Kumushi: *Root-Cause-Driven Automated Vulnerability Repair*. arXiv 2605.04251, 2026.
- *FalseCrashReducer: Mitigating False Positive Crashes in OSS-Fuzz-Gen Using Agentic AI*. arXiv 2510.02185, 2025.
- Google DeepMind. *Introducing CodeMender: an AI agent for code security*. October 6, 2025. <https://deepmind.google/blog/introducing-codemender-an-ai-agent-for-code-security/>

Multi-agent discovery and pentest

- Yang et al. *KernelGPT: Enhanced Kernel Fuzzing via Large Language Models*. arXiv 2401.00563, 2024–2025.
- Kong et al. *VulnBot: Autonomous Penetration Testing for a Multi-Agent Collaborative Framework*. arXiv 2501.13411, 2025.
- Ning et al. *MALF: A Multi-Agent LLM Framework for Intelligent Fuzzing of Industrial Control Protocols*. arXiv 2510.02694, 2025.
- *LLM-BSCVM: An LLM-Based Blockchain Smart Contract Vulnerability Management Framework*. arXiv 2505.17416, 2025.
- Nitin et al. *FaultLine: Automated Proof-of-Vulnerability Generation Using LLM Agents*. arXiv 2507.15241, 2025.
- Fang et al. *LLM Agents can Autonomously Hack Websites*. arXiv 2402.06664, 2024.
- Fang et al. *LLM Agents can Autonomously Exploit One-day Vulnerabilities*. arXiv 2404.08144, 2024.
- Fang et al. *Teams of LLM Agents can Exploit Zero-Day Vulnerabilities*. arXiv 2406.01637, 2024.

Agentic fuzzing

- *Agentic Fuzzing: Opportunities and Challenges*. arXiv 2605.10074, 2026.
- *Gentoo: Fuzzing with Agents? Generators Are All You Need*. arXiv 2604.01442, 2026.
- *GRIEF: Continuous Discovery of Vulnerabilities in LLM Serving Systems with Fuzzing*. arXiv 2605.11202, 2026.

Commercial systems

- XBOW. *The road to Top 1: How XBOW did it*. June 2025. <https://xbow.com/blog/top-1-how-xbow-did-it>
- Anthropic. *Claude Code Security Limited research preview*. February 2026. <https://www.anthropic.com/news/claude-code-security>
- Protect AI. *Vulnhuntr: Zero-shot vulnerability discovery using LLMs*. GitHub, October 2024. <https://github.com/protectai/vulnhuntr>

Benchmarks

- Bhatt et al. *CyberSecEval, CyberSecEval 2, CyberSecEval 3*. Meta AI Research, 2023–2024.
- Meta. *CyberSecEval 4 (including AutoPatchBench)*. 2025.
- *CVE-Bench*. arXiv 2503.17332, 2025.
- *ZeroDayBench: Evaluating LLM Agents on Unseen Zero-Day Vulnerabilities*. arXiv 2603.02297, 2026.
- *CyberGym* (Wang et al., 2025c, cited via ZeroDayBench).

Critical and skeptical literature

- Ullah et al. *LLMs Cannot Reliably Identify and Reason About Security Vulnerabilities (Yet?): A Comprehensive Evaluation, Framework, and Benchmarks*. NDSS 2024.
- *Everything You Wanted to Know About LLM-based Vulnerability Detection But Were Afraid to Ask*. arXiv 2504.13474, 2025.
- *LLM-based Vulnerability Detection at Project Scale: An Empirical Study*. arXiv 2601.19239, 2026.
- Steenhoek et al. *From Large to Mammoth: A Comparative Evaluation of LLM-based Vulnerability Detection*. NDSS 2025.
- Jin, Fu, and Li. *Good News for Script Kiddies? Evaluating Large Language Models for Automated Exploit Generation*. IEEE S&P Workshops 2025.
- Risse et al. *Robustness of LLM-based vulnerability detection*. (cited via Ullah et al.)

Agent-attack surface

- *Agent Security Bench (ASB): Benchmarking Vulnerabilities of LLM Agents*. ICLR 2025.
- *ChainFuzzer: Greybox Fuzzing for Workflow-Level Multi-Tool Vulnerabilities in LLM Agents*. arXiv 2603.12614, 2026.
- *AgentVigil* (cited via Dark Side of LLMs analysis, arXiv 2507.06850, 2025).

The Mythos disclosure

- Carlini, Cheng, Lucas, Moore, Nasr, and Anthropic Frontier Red Team. *Assessing Claude Mythos Preview's Cybersecurity Capabilities*. Anthropic Red Team, April 7, 2026.
<https://red.anthropic.com/2026/mythos-preview/>
- Anthropic. *Project Glasswing*. April 2026. <https://www.anthropic.com/glasswing>
- Hicks, Attridge, Janjeva, and Ashurst. *Claude Mythos: What Does Anthropic's New Model Mean for the Future of Cybersecurity?* CETaS Expert Analysis, April 2026.
- UK AI Security Institute. *Mythos Preview independent evaluation*. April 2026.
- *Claude Mythos and the AI Autonomous Offensive Threshold*. CSA Research Note, April 14, 2026.

Industry analysis and policy

- Bain & Company. *Claude Mythos and the AI Cybersecurity Wake-Up Call*. May 2026.
- TechTarget. *Claude Mythos Preview and the new rules of cybersecurity*. April 2026.
- Foreign Policy. *Anthropic's Claude Mythos Preview Changes Cyber Calculus*. April 20, 2026.
- HackerOne 2025 Annual Report; Bugcrowd 2026 "Inside the Mind of a Hacker."